

# Herramientas de la Java Development Kit

## Índice

|                                        |    |
|----------------------------------------|----|
| Herramientas Java .....                | 2  |
| Compilador .....                       | 2  |
| El entorno de ejecución .....          | 3  |
| Documentación de código: Javadoc ..... | 4  |
| Herramienta de empaquetado: Jar .....  | 8  |
| Herramienta de análisis: JConsole..... | 9  |
| Otras herramientas.....                | 10 |

## Herramientas Java

Ya se ha abordado una de las herramientas fundamentales del kit de desarrollo incluido con Java: el compilador. Pero este, como es de esperarse, no es la única herramienta que ofrece el kit. Veamos a continuación una lista, sino completa, pero extensiva, de dichas herramientas.

### Compilador

El compilador, toma los archivos de código fuente y los pasa a código que la máquina virtual puede utilizar. El “javac” (java compiler) hace lo que hacen todos los compiladores hacen: analizar la sintaxis del código fuente y, si es correcta, pasar el código a un estado “ejecutable”. De lo contrario, muestra los errores en la sintaxis. A diferencia de otros compiladores (siendo “jikes” el compilador de Java creado por IBM el mejor ejemplo), javac no hace optimizaciones sobre el código. Es decir, si repetimos variables, organizamos mal las sentencias de control, en fin, nuestro código es ineficiente, el resultado compilado será ineficiente. Por fortuna, la máquina virtual, antes de ejecutar el código (mediante el JIT visto en el módulo anterior) sí hace optimizaciones de todo tipo.

Sin contar con un IDE, la forma de utilizar javac es muy sencilla. Teniendo acceso al ejecutable del compilador, se debe llamar a javac seguido del archivo de código fuente a compilar. Por ejemplo:

```
>javac Auto.java
```

Si la clase Auto tiene una agregación con la clase Motor, por ejemplo, javac compilará la clase Motor de forma correspondiente. Es decir, javac compila las dependencias de cada clase según sea necesario. El archivo compilado resultante será “Auto.class”.

Por defecto, el compilador arroja los archivos compilados en el lugar donde se ejecuta. Alternativamente, se puede pasarle como argumentos desde donde tomar las fuentes y donde dejar los compilados, que es lo que generalmente hacen los IDE cuando usan el javac. En caso de usar librerías, es necesario pasarle al compilador la ubicación de estas

para evitar errores. El compilador javac tiene variedad de opciones, que pueden consultarse con:

```
>javac -help
```

O alternativamente:

```
>javac -X
```

Que brinda una serie “extendida” de configuraciones disponibles.

### El entorno de ejecución

Una vez compilados las fuentes, vamos a querer correr el programa. Para eso se utiliza el comando “java”, que llama al entorno de ejecución. Contrario a lo que se puede pensar *a priori*, no debemos indicarle el archivo a ejecutar, sino el nombre de la clase. Por ejemplo, si tomamos el caso anterior, donde compilamos Auto.java, y obtuvimos Auto.class, para “correr” el programa del auto debemos hacer:

```
>java Auto
```

Nótese como le pasamos el nombre de la clase a correr y no el nombre del archivo compilado.

Al igual que el compilador, el comando “java” acepta una serie de parámetros de configuración, todos opcionales, pero que sirven para tener un control más fino sobre la ejecución del programa. Los más importantes son:

- El parámetro classpath: el parámetro “classpath”, mediante el cual le indicaremos donde encontrar librerías, si el programa las necesita.
- Los parámetros Xms y Xmx: ambos controlan los límites de la memoria física que utilizará el programa al momento de ejecutarse. El comando Xms indica cuánta memoria física dispondrá el “heap” (ver módulo anterior) y el Xmx hasta cuánto podrá crecer (extenderse) para acomodar los objetos de los cuales necesite el programa. Si la ejecución, por la razón que sea, excede el valor de Xmx, se lanza un error (java.lang.OutOfMemoryError). Si no se especifican estos parámetros, se

establecen unos valores por defecto, de acuerdo con un algoritmo denominado “ergonómico”, descrito en el siguiente artículo (en inglés):  
<https://www.oracle.com/technetwork/java/ergo5-140223.html>

### Documentación de código: Javadoc

Otra herramienta muy común es la llamada “javadoc”. Javadoc analiza información incrustada en el código y genera documentación a partir de comentarios en el código fuente. El “javadoc” es una herramienta muy útil, ya que genera documentación en formato HTML y, genera también, un “sitio” con la misma, dividida en <frames> para facilitar la navegación de esta.

A continuación, presentaremos una breve reseña de las instrucciones necesarias para poder generar el “javadoc”.

Como vimos en el módulo de sintaxis, los comentarios en java pueden hacerse con la doble barra “//” o si era un comentario de múltiples líneas, se podía encerrar entre “/\*” y “\*/”. Para poder disparar la generación de “javadoc”, es necesario incluir “comentarios especiales”. Estos no son más que comentarios regular, pero comienzan siempre con “/\*\*”. Veamos un ejemplo:

```

package paquete1;

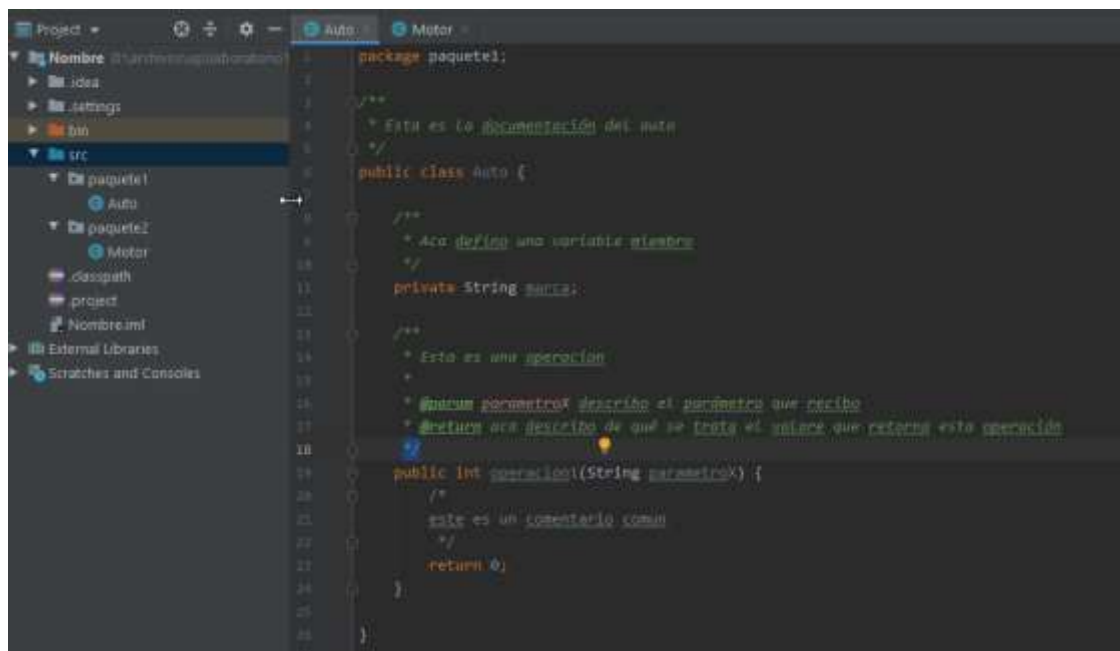
/**
 * Esta es la documentación del auto
 */
public class Auto {

    /**
     * Aca defino una variable miembro
     */
    private String marca;

    /**
     * Esta es una operación
     *
     * @param parametroX describo el parámetro que recibo
     * @return aca describo de qué se trata el valor que retorna esta operación
     */
    public int operacion1(String parametroX) {
        /*
         * este es un comentario comun
         */
        return 0;
    }
}

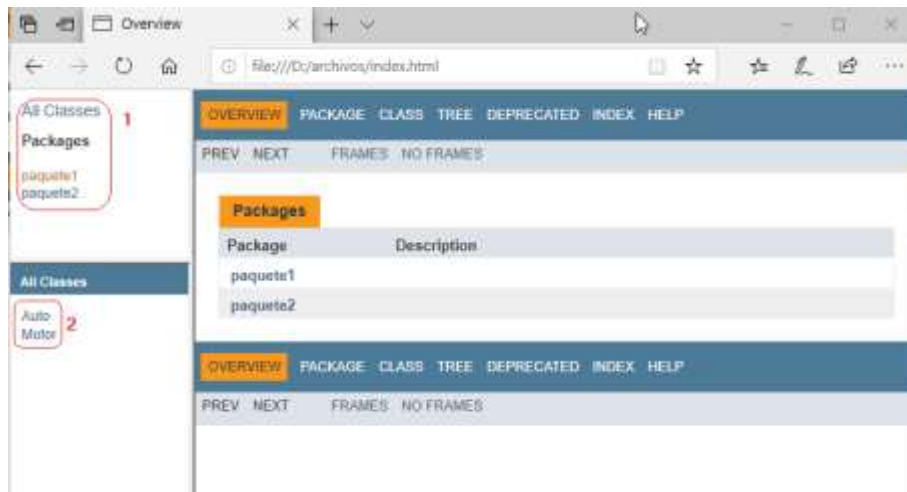
```

Nótese como el IDE “pinta” los “comentarios de javadoc” con un color distinto de los “comentarios de código”. Para poder generar la documentación, utilizaremos el comando “javadoc”. Este funciona de manera similar a “javac”, en el sentido que le debemos pasar la clase con los comentarios y este irá recorriendo las otras clases que esta primera utilizar según sea necesario. Durante el recorrido, irá generando la documentación. Por supuesto, generar la documentación manualmente de todo un sistema (como compilar) puede ser muy complejo o tedioso. Lo mejor, hoy en día, es utilizar un IDE para estas tareas. Veamos cómo quedaría la documentación luego de correr el “javadoc” en un proyecto con esta estructura:

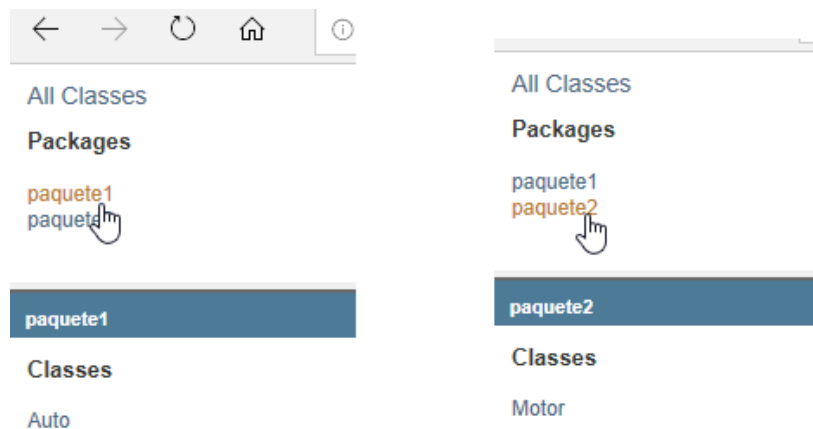


```
1 package paquete1;
2
3 /**
4  * Esta es la documentación del auto
5  */
6 public class Auto {
7
8     /**
9      * Aca defino una variable miembro
10     */
11     private String marca;
12
13     /**
14      * Esta es una operacion
15      *
16      * @param parametro describe el parametro que recibo
17      * @return una descripción de qué se trata el valor que retorna esta operacion
18     */
19     public int operacion(String parametro) {
20         /*
21          * este es un comentario comun
22          */
23         return 0;
24     }
25 }
26 }
```

Y el sitio con la documentación resultante:



Acá, en el punto marcado como “1” se puede ver cómo podemos navegar entre los paquetes (o de selecciona “All Classes” se muestran todas, sin filtrar) y en el punto “2” están las clases de cada paquete (o todas las clases en su defecto). Por ejemplo:



Y luego, al seleccionar una clase, podemos ver cómo están todos los “comentarios especiales” que pusimos en el código:

All Classes
Packages
paquete1
paquete2

All Classes
Auto
Motor

OVERVIEW
PACKAGE
CLASS
TREE
DEPRECATED
INDEX
HELP

PREV CLASS
NEXT CLASS
FRAMES
NO FRAMES

SUMMARY
NESTED
FIELD
CONSTR
METHOD
DETAIL: FIELD
CONSTR
METHOD

paquete1

### Class Auto

java.lang.Object
paquete1.Auto

---

```
public class Auto
extends java.lang.Object
```

Esta es la documentación del auto

#### Constructor Summary

Constructors

Constructor and Description

Auto{}

#### Method Summary

All Methods
Instance Methods
Concrete Methods

| Modifier and Type | Method and Description                                           |
|-------------------|------------------------------------------------------------------|
| int               | operacion1(java.lang.String parametroX)<br>Esta es una operación |

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

#### Constructor Detail

Auto

```
public Auto()
```

#### Method Detail

operacion1

```
public int operacion1(java.lang.String parametroX)
```

Esta es una operación

Parameters:

parametroX - describo el parámetro que recibo

Returns:

aca describo de qué se trata el valor que retorna esta operación

OVERVIEW
PACKAGE
CLASS
TREE
DEPRECATED
INDEX
HELP

PREV CLASS
NEXT CLASS
FRAMES
NO FRAMES

SUMMARY
NESTED
FIELD
CONSTR
METHOD
DETAIL: FIELD
CONSTR
METHOD

Herramienta de empaquetado: Jar

El comando “jar” sirve para empaquetar clases compiladas en forma de librerías. El archivo generado es un archivo con extensión “.jar”. El formato de empaquetado (y

UP

© Universidad de Palermo. Prohibida la reproducción total o parcial de imágenes y textos.

8



compresión, si se configura) es “zip” estándar, solo que cambia la extensión (de hecho, podemos abrir un jar con el “WinZip” o cualquier herramienta similar). Asimismo, se puede utilizar el comando “jar” para empaquetar todo un programa, incluso junto con la clase principal de este, la que se utiliza para iniciar el programa. Entonces, podríamos generar un entregable con todo el programa para que pueda ejecutarse. Para hacer una analogía con Windows, sería como generar un “.exe” con todo nuestro programa. De hecho, con la herramienta “jar” se puede armar el paquete y al mismo tiempo indicar cuál es la clase para ejecutar. Con esto, y usando la misma herramienta “jar”, podemos mandar a ejecutar el paquete:

#### Compilar la clase

```
>javac Auto.java
```

#### Empaquetar el programa Auto, generando un “ejecutable” llamado MiProgramaAuto

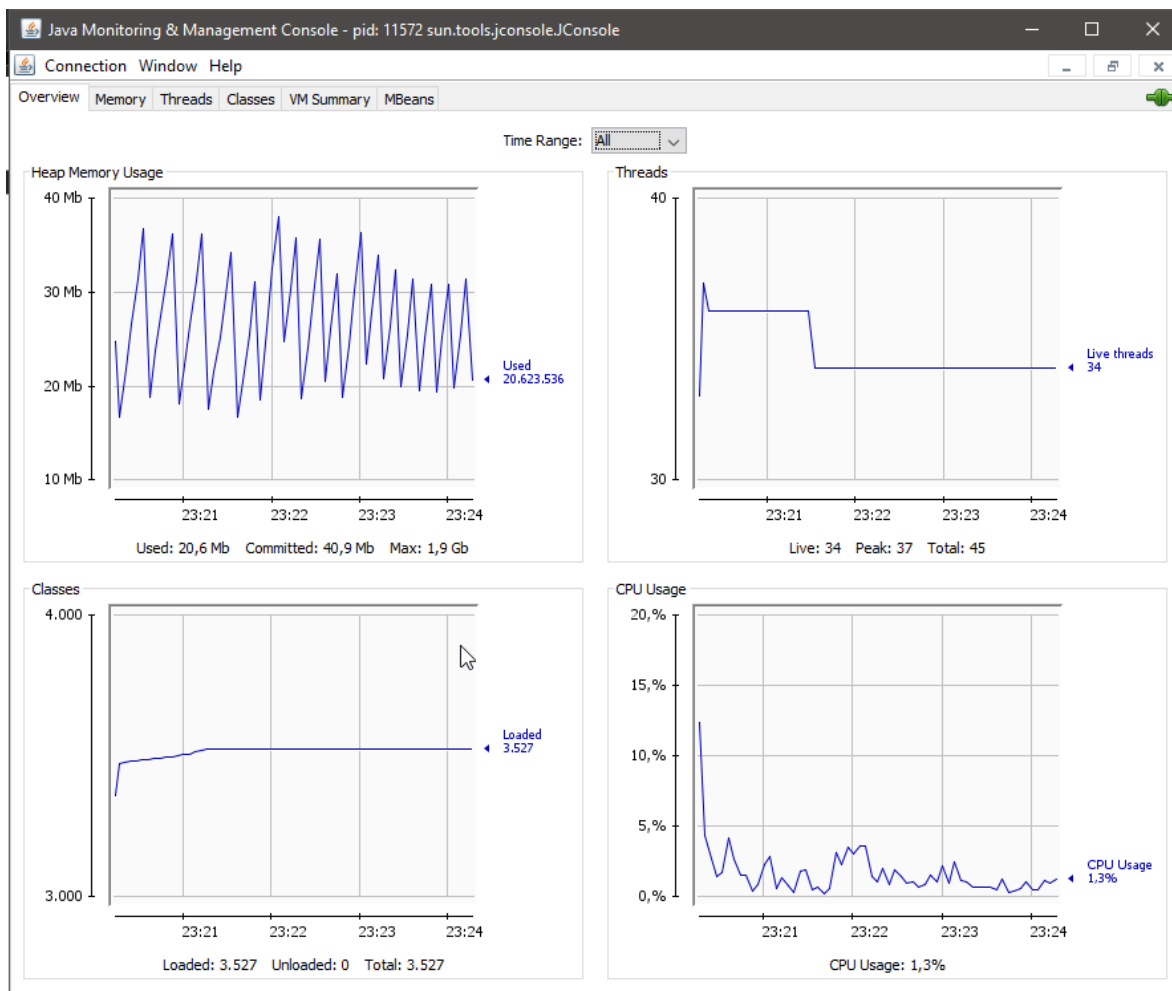
```
>jar -cvf MiProgramaAuto.jar Auto.class
```

#### Ejecutar el programa, utilizando la herramienta “jar”

```
>jar MiProgramaAuto
```

#### Herramienta de análisis: JConsole

Es una herramienta de profiling. Con ella podemos analizar aspectos básicos de la performance de nuestro programa. Es una herramienta gráfica que se conecta a un proceso java (un programa java en ejecución) y muestra información sobre el uso de memoria (heap), de procesador, de hilos de ejecución, etc. Si bien es una herramienta básica, puede ser muy útil para diagnosticar problemas con nuestra aplicación, analizando los síntomas y la información reportada. Acá podemos ver la pantalla principal de JConsole, analizando la ejecución de sí mismo (como es de esperarse, JConsole, está programado en Java).



## Otras herramientas

La lista de herramientas es extensa, hay varias herramientas más, aunque escapan al scope de la materia. Podemos mencionar herramientas para generar pares de claves publica/privada, firmar clases, firmar paquetes (jars generados), generar web services, etc.